

aptana® Jaxer® The world's first Ajax server

Home Quick Start **Guide** Tutorials API Screencasts Forums Download

Jaxer Guide

- Introduction
- Setup
- Develop
 - Application Level Concepts
 - Persistent Storage
 - Communications
 - Web Services
 - Web Requests
 - URLs
 - Email
 - Sockets
 - Utilities
 - Advanced Topics
- Debug

Files

File Basics

Simple Files vs. File Objects

Whether you create an object or use the static methods, Jaxer creates a File object, so the key reason to use an object boils down to convenience. If you'll only be calling one or two methods against the file then static is simpler; for more complex work create an object and store it in a variable for reuse.

File Modes

When opening a file in Jaxer you pass a file mode to open to control how the file will be accessed. The options for this are show below. Use the binary mode strings when the data stored is not regular text; this is sometimes referred to as 'binary safe' file access.

Demonstrating the Different File Modes

```
01. // The following lines assume use of a File
    // object called myfile
02. var myfile = new File('somefile.ext');
03. myfile.open('r'); // open for reading
04. myfile.open('w'); // open for writing, overwrites contents
05. myfile.open('a'); // open for appending data, new data is added to the file
06.
07. myfile.open('rb'); // open binary file for reading
08. myfile.open('wb'); // open binary file for writing, overwrites contents
09. myfile.open('ab'); // open binary file for appending data, new data is added
```

Contents

[File Basics](#)
[Simple File Access](#)
[Using File Objects](#)
[File IO](#)
[Uploading Files](#)
[Downloading Files](#)

API Links

[Jaxer.File](#)
[Jaxer.Request](#)
[Jaxer.Request.FileInfo](#)

Simple File Access

File Operations

Create

Use Jaxer.File.touch(path) to create a file if it doesn't exist and set the last modified timestamp if it does:

```
1. Jaxer.File.touch('myfile.txt');
```

Truncate

Use Jaxer.File.truncate(path) to empty the file of its current contents:

```
1. Jaxer.File.truncate('myfile.txt');
```

Delete

Jaxer.File.remove(path) deletes the file from the filesystem:

```
1. Jaxer.File.remove('myfile.txt');
```

Check for Existence

Use the Jaxer.File.exists(path) static method:

```
1. if (Jaxer.File.exists('myfile.txt'))
2. {
3.     // myfile.txt exists
4. }
5. else
6. {
7.     // myfile.txt does not exist
8. }
```

Copy

Jaxer.File.copy(source,destination) will copy the file to a new location, leaving the original file untouched. It will, however, overwrite the destination file if it exists and your application has sufficient privileges.

```
1. Jaxer.File.copy('myfile.txt', 'myfile.bak');
```

Move

Jaxer.File.move(source,destination) will move the file to a new location and the original file will be deleted. It will, however, overwrite the destination file if it exists and your application has sufficient privileges.

```
1. Jaxer.File.move('myfile.txt', 'myfile.bak');
```

Backup

Jaxer.File.backup(path) creates a uniquely named copy of the original file and returns the path to the newly created copy:

```
1. var backupFilePath = Jaxer.File.backup('myfile.txt');
```

File Attributes

Size

Jaxer.File.size(path) returns the size of the file in bytes:

```
1. var size = Jaxer.File.size('myfile.txt');
```

Extract parts of the filepath/name

- Filename portion of the path: Jaxer.File.filename(path)
- File extension: Jaxer.File.extension(path)
- Absolute path to the file: Jaxer.File.absolutePath(path)
- Path to the containing folder: Jaxer.File.parentPath(path)

```
1. var filename= Jaxer.File.filename('c:\\folder1\\folder2\\myfile.txt');
2. var filename= Jaxer.File.extension('c:\\folder1\\folder2\\myfile.txt');
3. var absolutePath = Jaxer.File.absolutePath('c:\\folder1\\folder2\\myfile.txt');
4. var parentPath = Jaxer.File.parentPath('c:\\folder1\\folder2\\myfile.txt');
```

Last Modified Date

Use Jaxer.File.dateModified(path):

```
1. var dateModified = Jaxer.File.dateModified('myfile.txt');
2. // returns 'Mon Mar 03 2008 19:13:17 GMT-0800 (Pacific Standard Time)'
```

Access Permissions

Jaxer.File.chmod(path,permissions) gets and sets file permissions; the path parameter is required, the permissions parameter is optional. Passing the permissions value is the setter, passing just a path is a getter. The permission value provided/returned is an octal number, in JavaScript this is a numeric literal with a leading 0.

```
1. Jaxer.File.chmod('myfile.txt',0400); // read only
2. var permissions = Jaxer.File.chmod('myfile.txt'); // -> '0400' (Windows: 0444)
```

CRC32 Checksum

Use Jaxer.File.checksum(path):

```
1. var checksum = Jaxer.File.checksum('myfile.txt');
```

File Content Operations

Get Contents

Jaxer.File.read(path) retrieves the entire contents of the file into a JavaScript variable:

```
1. var contents = Jaxer.File.read('myFile.txt');
```

Append Content

Jaxer.File.append(path,content, content2, ..., contentN) will add the content to the end of the file and if the file doesn't exist it will be created.

```
1. Jaxer.File.append('myfile.txt','line1','line2');
```

Append a Line of Content

Jaxer.File.appendLine(path,content, content2, ..., contentN) will add the content to the end of the file, appending a new line after each passed content parameter, and if the file doesn't exist it will be created.

```
1. Jaxer.File.appendLine('myfile.txt','line1','line2');
```

Using File Objects

Creating a File Object

Create a new Jaxer.File(pathToFile) object; the parameter must include the full path to the referenced file system object or else the call will throw a 'NS_ERROR_FILE_UNRECOGNIZED_PATH' error. Note that creating a file object does not automatically create the equivalent object on the file system.

```
1. // use Jaxer.Dir.resolve(path) to avoid error
2. var filePath = Jaxer.Dir.resolve('myFile.txt');
3. var myFile = new Jaxer.File(filePath);
```

File Information

- Exists:

```
1. // if the file doesn't exist create an empty file.
2. if (!myFile.exists())
3. {
4.     myFile.create();
5. }
```

- Size: var size = myFile.size;
- Name: var leaf = myFile.leaf;
- Extension: var ext = myFile.ext;
- Path (includes name): var path = myFile.path;
- Parent Path (excludes name): var parentPath = myFile.parentPath;
- Last Modified Date: var dateModified = myFile.dateModified;
- File URL: var URL = myFile.URL;
- MIME Type: var mimeType = Jaxer.File.mimeType(resolvedPathObject);
- Attributes: (shown in the next block)

```
1. var isDir = myFile.isDir(); // false
2. var isFile = myFile.isFile(); // true
3. var isExec = myFile.isExec(); // false
```

```

4. var isSymlink = myFile.isSymlink(); // false
5. var isWritable = myFile.isWritable(); // true
6. var isReadable = myFile.isReadable(); // true
7. var isHidden = myFile.isHidden(); // false
8. var isSpecial = myFile.isSpecial(); // false

```

Managing Files

Create

Invoke the create() method on a file object:

```

1. var myFile = new Jaxer.File(Jaxer.Dir.resolve('myFile.txt'));
2. if (!myFile.exists())
3. {
4.     myFile.create();
5. }

```

Delete

Use the remove() method. Once you delete the file, some methods previously available on the file object will throw exceptions if accessed.

```

1. // if the file exists delete it
2. if (!myFile.exists())
3. {
4.     myFile.remove();
5. }

```

Truncate

Use truncate() to empty the file. After truncating, the file size will be zero bytes.

```

1. // if the file exists truncate it
2. if (!myFile.exists())
3. {
4.     myFile.truncate();
5. }

```

Copy

The copy() method will copy the file to a provided file location; it throws an exception if the destination file already exists, which is different from the static file copy method's behavior.

```

1. if (myFile.exists() && myFile.isReadable())
2. {
3.     var to = Jaxer.Dir.resolve('copy.txt');
4.     myFile.copy(to);
5.
6.     to = Jaxer.Dir.combine(Jaxer.System.tempFolder, myFile.leaf);
7.     myFile.copy(to);
8. }

```

File IO

Reading from Files

File Reading Setup

```

1. var readmePath = Jaxer.Dir.combine(Jaxer.System.executableFolder, 'README-JAXER');
2. var readme = new Jaxer.File(readmePath);

```

Entire File

Read the entire contents of that file into a string, specifying the 'r' (read) access mode to the open method:

```

1. if (readme.exists() && readme.isReadable())
2. {
3.     readme.open('r');
4.     var contents = readme.read();
5.     Jaxer.Log.info(contents);
6.     readme.close();
7. }

```

Note: In a real application you would check that the file exists and is readable before opening it. Also, by default files are opened for read, so in this case, you could simply have omitted the access mode.

As an Array of Lines

The readAllLines() method returns the contents of a file as an array of individual lines. This method uses platform-specific line terminators when splitting the contents, and so is safer than using the String split("\n") method.

```

1. if (readme.exists() && readme.isReadable())
2. {
3.     readme.open('r');
4.     var contents = readme.readAllLines();
5.     Jaxer.Log.info(contents.length);
6.     readme.close();
7. }

```

One Character at a Time

The read() method takes a parameter representing the number of bytes to read from the file so to read the file one byte at a time, pass the value 1 to read() inside a loop until reaching file's end.

```

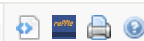
01. if (readme.exists() && readme.isReadable())
02. {
03.     readme.open('r');
04.     while ( ch = readme.read(1) )
05.     {
06.         Jaxer.Log.info(ch);
07.     }
08.     readme.close();
09. }

```

Writing to Files**Creating an Empty File**

Use the create() method. This code will create an empty file named 'myFile.txt' in the same folder as the current page.

```
1. if (!myFile.exists())
2. {
3.     myFile.create();
4. }
```

**Appending Content**

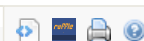
Appending to a file is done by opening a file in 'append mode' and writing data to that file; the 'a' parameter indicates the file should be opened in append mode.

```
1. if (myFile.exists() && myFile.isWritable())
2. {
3.     myFile.open('a');
4.     myFile.write("some text to write");
5.     myFile.close();
6. }
```

**Replacing the Content**

Replace the contents of a file (or overwriting) by opening it in write mode, writing data to the file and closing it (this last step is important!).

```
1. if (myFile.exists() && myFile.isWritable())
2. {
3.     myFile.open('w');
4.     myFile.write("some text to write");
5.     myFile.close();
6. }
```

**Uploading Files**

Uploading files to Jaxer from a web form is a straightforward process. All you need is a form containing an input element of type upload and an html page to receive the submitted form.

HTML Upload Form

```
01. <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/
02. <html>
03. <head>
04.     <meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1"/>
05.     <title>Upload </title>
06. </head>
07. <body>
08.     <form id="upload" action="submissions.html" method="post" enctype="multipart/form-data">
09.         <input id="fileUpload" type="file" name="myFile1"><br>
10.         <input id="fileUpload" type="file" name="myFile2"><input type="submit" value="upload">
11.     <p>
12.         'upload' Sends the selected file(s) to the server.
13.     </p>
14. </form>
15. </body>
16. </html>
```

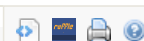
**Receiving File Uploads**

This code goes into the page which receives the data from the submitted form; the posted data is available within the Jaxer.request object. We're specifically interested in Jaxer.Request.files, an array of Jaxer.Request.FileInfo objects, which has descriptive properties such as fileName, contentType and fileSize.

```
1. var message = "";
2.
3. for( fileCount=0; fileCount<jaxer.request.files.length; fileCount++) { var
  ↳ fileinfo="Jaxer.Request.files[fileCount];" var
    destinationfilepath="Jaxer.Dir.resolve(fileinfo.originalFileName);"
    fileinfo.save(destinationfilepath); message += "<BR>" + [ "saved to : " +
    destinationfilepath , "original filename : " + fileinfo.originalfilename , "temp filename
    : " + fileinfo.tempfilename , "contenttype : " + fileinfo.contenttype , "size : " +
    fileinfo.filesize ].join("<br>");
4.
5. }
6. document.write(message);
```



```
01. // outputs something similar to:
02. Saved to C:\aptana\JaxerDev\public\work\bar.txt
03. original filename : bar.txt
04. temp filename : C:\aptana\JaxerDev\tmp\tmp
05. contentType : text/plain
06. size : 15754
07. Saved to : C:\aptana\JaxerDev\public\work\foo.txt
08. original filename : foo.txt
09. temp filename : C:\aptana\JaxerDev\tmp\tmp-1
10. contentType : text/plain
11. size : 816
```

**Downloading Files**

```
01. // Read in the contents of an RTF and send it to the user
02. var path = Jaxer.Dir.resolve("example1.rtf", Jaxer.request.currentFolder);
03. var file = new Jaxer.File(path);
04. file.open('rb');
05. var contents = file.read();
06. file.close();
07. Jaxer.response.setContentsBinary(contents, "text/rtf");
08.
09. // Grab an image from the current directory and send it to the user
10. var path = Jaxer.Dir.resolve("image.png", Jaxer.request.currentFolder);
```



```
11. Jaxer.response.setContentsFile(path);
```